SyncSpace: Explainable Campus Project and Teammate Matching

Project Proposal for UCS503P

Submitted to: Raghav B. Venkataramaiyer

Thapar Institute of Engineering and Technology

Aryan Bansal	**Yashika**	**Rumani**
Roll No: 1024030690	Roll No: 1024030680	Roll No: 1024030678

August 17, 2026

1 Higher-order goal

...secondary framing

SyncSpace supports equitable access to practical peer learning. Students should be able to contribute to meaningful projects because their skills and availability fit the work, not only because they already know the project owner. The broader goal is a more inclusive campus project ecosystem; the immediate engineering objective is a measurable, deployable workflow for discovering projects, forming teams, and beginning collaboration.

2 Time-to-value

...primary heuristic

- Deliver a meaningful vertical slice quickly: verified sign-in, profile creation, project discovery or publishing, application, owner decision, and accepted-team workspace.
- Use short weekly iterations and production feedback rather than waiting for one final release. Each increment is built, tested, deployed, and reviewed.
- Define first value as a student submitting one valid application or publishing one valid project; target a median time of 10 minutes or less during the pilot.

3 Problem Statement

Thapar students commonly form project teams through class groups, personal networks, and direct messages. These channels are fast for a small circle but do not provide a consistent way to compare project scope, required skills, weekly commitment, capacity, deadline, or application status. Common pain points include:

- **Fragmented discovery:** opportunities are scattered across messages and informal conversations, so suitable students may never see them.
- **Network bias:** project owners tend to recruit people they already know, while capable students outside that network have less visibility.
- **Unclear fit:** a project card or message rarely explains why a student's skills, interests, and availability match the work.
- **Manual coordination:** owners repeatedly explain the brief, track applicants manually, exchange contacts, and move accepted members to another tool.
- **Privacy risk:** resumes, emails, and team information can be exposed without a clear ownership or membership boundary.

Why this matters now (contextual relevance): Thapar students already undertake course, club, hackathon, research, and portfolio projects. A controlled 50-student pilot can produce visible results within the semester and directly test whether a structured workflow reduces coordination effort.

4 Proposed Solution

...Software Proposition

4.1 Overview

Build a web-based campus project and teammate matching platform with the following components:

- **Verified institutional account:** server-verified Google identity restricted to the configured Thapar domain; one account can both join and lead projects.
- **Student profile:** skills with proficiency, interests, weekly availability, short biography, and optional private PDF resume with review-before-save skill extraction.
- **Project publishing and discovery:** structured briefs with domain, required and preferred skills, capacity, commitment, deadline, and lifecycle controls.
- **Explainable matching:** deterministic ranking that exposes skill coverage, interest alignment, availability, commitment fit, and missing skills.
- **Application and query workflow:** students can ask focused asynchronous project questions, apply, and track decisions; owners can respond and accept or reject applicants.
- **Protected collaboration:** accepted members can view teammate contacts and profiles and use a shared task workspace; owners and the configured administrator receive scoped lifecycle controls.

4.2 Core workflow

1. A student signs in with a verified institutional identity and completes a structured profile.
2. The student searches by skill or receives ranked projects with a visible match explanation.
3. The student reviews the project brief, raises a query if needed, and submits an application.
4. The owner reviews the applicant and accepts or rejects the application. Acceptance atomically creates team membership while enforcing capacity.
5. The owner and accepted collaborators open the shared workspace, view permitted contact details, and assign or update tasks.

4.3 Operational constraints for a campus pilot

- Keep the interface responsive and usable on typical student phones and laptops.
- Avoid dependence on novel prediction or research algorithms; use deterministic, auditable scoring and mature infrastructure.
- Keep resumes in private object storage and expose contact data only through authorized relationships.
- Deploy within educational or free-tier limits while documenting cold starts, quotas, and operational boundaries.

5 Solution Approach

...Engineering focus

This is a software engineering project. The emphasis is on requirements, architecture, authorization, transactions, testing, deployment, observability, and user evaluation rather than claiming that an algorithm can predict team success.

5.1 Explainable matching

...non-research

Project recommendations use a deterministic weighted model:

- 50% skill coverage, with required skills weighted above preferred skills.
- 20% domain and interest alignment.
- 15% availability fit and 15% commitment fit.

- Hard eligibility checks exclude closed, expired, full, already-applied, or already-joined projects.

The response exposes component scores, covered skills, gaps, and eligibility reasons. The pure scoring module is independent of PostgreSQL so its normalization, ordering, and edge cases can be unit-tested.

5.2 Web architecture

...web-based solution

SyncSpace uses a three-tier architecture:

- **Frontend:** React 19, React Router, and Vite provide the public landing page, project discovery, owner controls, profiles, and workspace interface.
- **Backend API:** Express 5 applies Zod validation, JWT authentication, exact-domain Google verification, ownership checks, membership checks, and transactional workflows.
- **Data and files:** PostgreSQL stores tenant-scoped relational data; a private S3-compatible Supabase bucket stores resumes; parameterized SQL and numbered migrations keep changes reproducible.

The deployed client is hosted on Appwrite Sites, the API on Render, and PostgreSQL plus object storage on Supabase. A `college_id` boundary is retained so the design can later support additional institutions without expanding the current pilot scope.

5.3 CI/CD and fast delivery enabler

GitHub Actions supports the short iteration cadence by running the following pipeline on relevant changes:

- Install the locked dependency graph and validate production configuration.
- Start PostgreSQL, apply every numbered migration, and seed repeatable test data.
- Run server, client, unit, component, and SQL-backed integration tests.
- Build the production React bundle, start the API, and execute the documented smoke journey.
- Upload the client build artifact; accepted main-branch changes trigger the configured Render and Appwrite deployments.

The pipeline begins with practical continuous integration and expands delivery automation only where it shortens feedback and improves release confidence.

6 Evaluation Criterion

...measurable and attributable

Success is measured using observed pilot data. Targets are proposal criteria and will not be reported as achieved until participant sessions are complete.

6.1 Primary evaluation metric

Time to first value: elapsed time from successful sign-in until the participant submits one valid application or publishes one valid project.

- **Target:** median time to first value <= 10 minutes during the 50-student pilot.
- **Attribution:** derived from timestamped workflow events and a pseudonymous participant session record.

6.2 Secondary evaluation metrics

- **Onboarding completion:** at least 80% sign in and save a usable profile without facilitator intervention.
- **First-value completion:** at least 75% successfully apply or publish.
- **Explanation comprehension:** at least 75% identify the largest component of one recommendation.
- **Owner decision and workspace handoff:** at least 80% of assigned owners decide an application and 80% of accepted members locate contact/profile/task controls.
- **Critical-flow reliability:** at least 95% successful critical API responses, excluding deliberate validation errors.
- **Authorization and usefulness:** zero confirmed unauthorized disclosures and median usefulness >= 4/5.

6.3 Pilot validation plan

...high-level

- Recruit at least 50 current Thapar students across at least two course or lab groups and allow them to use their own phones or laptops.
- Session A covers sign-in, profile, and either discover/apply or publish. Session B covers owner decision and accepted-team workspace handoff.
- Use participant codes P01-P50 and store only completion, timing, viewport class, error code, comprehension, and a short sanitized feedback response.
- Report actual numerators, denominators, medians, failures, device mix, and deviations from the proposal.

7 Scalability

...with foresight

1. **Scalable in principle:** the stateless API, relational constraints, independent object storage, pagination-ready queries, and explicit tenant boundary support growth beyond one class. Database indexes and caching can be added from measured bottlenecks rather than speculation.
2. **Operationally deployable:** the client, API, database, and object storage are independently hosted on mature managed services. Free tiers can be upgraded without replacing the application architecture, and additional API instances can be introduced behind a load balancer.

8 Engine availability heuristic

The core solution is supported by mature engines already available to the team:

- React, React Router, and Vite for a responsive web client.
- Express, Zod, JWT, bcrypt, and Google identity verification for API and authentication workflows.
- PostgreSQL for transactional relational data and tenant boundaries.
- S3-compatible object storage for private resume files.
- Node test runner, Vitest, Testing Library, Supertest, and GitHub Actions for automated verification.
- Appwrite Sites, Render, and Supabase for managed deployment.

These engines allow the team to focus on workflow correctness, security boundaries, scalability, and measurement rather than building infrastructure or conducting algorithm research.

9 Project scope and deliverables

...iteration-friendly

9.1 Initial deliverable

...first iteration

- Repository foundation, database migrations, test harness, and health endpoints.
- Verified sign-in, student profile, skills, availability, and resume adapter.
- Project publishing, discovery, explainable scoring, and application creation.
- CI pipeline for migrations, seed, tests, build, smoke journey, and artifact upload.

9.2 Subsequent deliverables

- Owner application tracking with accept/reject controls and capacity-safe team creation.
- Unified account navigation, protected teammate contacts/profiles, and shared tasks.
- Project editing/deletion, scoped administrator controls, and audit records.
- Project query-and-response workflow, public skill search, mobile refinement, and consistent production styling.
- Operational hardening, 50-student pilot, measured evaluation, and final report.

10 Risks and mitigations

- **Privacy and authorization:** minimize collected data, keep resumes private, verify domain server-side, and test owner/member/tenant boundaries negatively.

- **Insufficient project supply:** recruit project owners before the pilot and monitor whether common skills return meaningful results.
- **Free-tier cold starts and quotas:** separate cold and warm timing, monitor usage, and document service limits.
- **Inappropriate or low-quality content:** provide owner lifecycle controls and tenant-scoped administrator deletion with a durable reason and audit trail.
- **Evaluation bias:** use a defined task script, pseudonymous records, actual denominators, and explicit reporting of convenience-sample limitations.
- **Scope growth:** keep payments, public social feeds, native apps, and production encrypted chat outside the assessed core.
- **Deployment regressions:** require repeatable CI checks, readiness endpoints, smoke tests, and short rollback-ready releases.

11 Summary

SyncSpace is a locally relevant, deployable software-engineering proposition for improving campus project formation. It delivers value through one verified account, structured project discovery, transparent matching, controlled application decisions, protected contact exchange, and a collaboration handoff. The project uses mature engines, a PostgreSQL-backed architecture, and automated build-test-release feedback rather than research-dependent novelty. Its success will be judged through a reproducible 50-student pilot measuring time to first value, workflow completion, explanation comprehension, reliability, usefulness, and authorization safety.